Assignment 2

Questions

1. Why does BFS ignore the distances stored in the graph even though they are available to it?

BFS try to miniminises the number of edges, not the km cost because it doesn't uses the distance. Every edge is the same step. It explores with FIFO queue.

2. Which two lines change when bfs is turned into dfs, and which of them makes DFS order dependent?

Line : frontier = deque([start]) in BFS vs frontier = [start] in DFS.
Line : frontier.popleft() (FIFO) in BFS vs state = frontier.pop() (LIFO) in DFS
DFS depends on the order of neighbors because it explores the last added node first. So, changing the order of neighbors could change the path explored.

3. UCS tests for the goal when a state is popped from the heap. What could go wrong if it tested when the state was first generated?

A cheaper path could exit and could be not seen by UCS. UCS could find a path that is not optimal. By checking the goal when a node is removed from the priority queue, UCS ensures that the cheapest path is found.

4. Lab1reports *expanded* and *runtime_ms*. Which one should you use to compare BFS and UCS with a classmate's run on a different laptop, and why?

The number of expanded nodes is a better way to compare search algorithms. Execution time can change depending on the machine or background processes.

5. Which quantity does GBFS ignore that A⋆ uses, and which failure in Lab2 does that explain?

GBFS only considers the heuristic value h and ignores the cost already traveled g.

It may choose a path that looks closer to the goal but is actually longer or more expensive. It's the case in Lab 2. In Lab 2, KUL → KEF: GBFS returns KUL → LHR → KEF at 12 500 km (2 expansions), while the optimum is 11 461 km (UCS and A*).

6. Frontier states *X* and *Y* have *(g, h) = (8,2)* and *(3,6)*. Which does UCS expand first? GBFS? A⋆?

- UCS: Y
- GBFS: X
- A*: Y

7. Why must *h* be expressed in the same units as the edge costs?

A* calculates f = g + h to know the node to explore. So, both g and h must be expressed in the same units.

8. Lab2 computes $h^\star$ by running Dijkstra on the reversed graph from the goal. Why reversed?

Since the graph is directed, the shortest path from A to B is not necessarily the same as from B to A. Reversing the edges allows Dijkstra to calculate the shortest distance from each airport to the goal in the original graph.

9. Weighted A⋆ with *w = 1.5* changed the cost for KUL → KEF but not for SIN → EZE. What property of the SIN → EZE route explains this?

The optimal route SIN → EZE remain attractive even when the heuristic is given more weigh. The SIN → EZE route has relatively few detours, meaning the straight-line distance is close to the actual travel distance.